

```
#!/usr/bin/env python3
"""
BruteForce Engine – God Tier Attack Core
Intelligent multi-modal attack: Markov chains · rule
engine · hybrid pipeline

Modes
-----
    brute    → exhaustive combinatorial (guaranteed,
slow, pairs with generator.py output)
    wordlist → dictionary attack from file
    markov   → statistical pattern generation (fast for
human-like targets)
    hybrid   → wordlist → mutations → Markov → brute
(recommended)

Examples
-----
    python engine.py --hash
5f4dcc3b5aa765d61d8327deb882cf99
    python engine.py --hash <hash> --mode hybrid --
wordlist rockyou.txt
    python engine.py --hashfile targets.txt --algo
sha256
    python engine.py --hash <hash> --mode brute --
wordlist combinations.txt # use generator.py output
    python engine.py --benchmark
"""

from __future__ import annotations

import argparse
import hashlib
import itertools
import json
import os
import queue
import random
```

```

import string
import sys
import threading
import time
from collections import defaultdict
from dataclasses import dataclass, field
from pathlib import Path
from typing import Callable, Dict, Generator, List,
Optional, Set

#
=====

# 1. HASH LAYER - md5 · sha1 · sha256 · sha512 ·
ntlm
#
=====

_ALGORITHMS: Dict[str, Callable[[str], str]] = {
    "md5": lambda p:
hashlib.md5(p.encode()).hexdigest(),
    "sha1": lambda p:
hashlib.sha1(p.encode()).hexdigest(),
    "sha256": lambda p:
hashlib.sha256(p.encode()).hexdigest(),
    "sha512": lambda p:
hashlib.sha512(p.encode()).hexdigest(),
    "ntlm": lambda p: hashlib.new("md4",
p.encode("utf-16-le")).hexdigest(),
}

_LEN_MAP: Dict[int, str] = {32: "md5", 40: "sha1", 64:
"sha256", 128: "sha512"}

def auto_detect(h: str) -> Optional[str]:
    """Guess algorithm from hash string length."""
    return _LEN_MAP.get(len(h.strip()))

```

```

def get_hash_fn(algo: str) -> Callable[[str], str]:
    fn = _ALGORITHMS.get(algo.lower())
    if not fn:
        raise ValueError(f"Unknown algorithm '{algo}'.
Valid: {sorted(_ALGORITHMS)}")
    return fn

#
=====
=====
# 2. RULE ENGINE - mutations applied to every base
candidate
#
=====
=====

_LEET = str.maketrans("aeiost", "@3!0$+")

def mutate(word: str) -> List[str]:
    """
    Apply common password transformation rules to a
    base word.
    Covers: leet-speak, capitalization, reversal,
    common suffixes/prefixes,
    doubling, and year appending.
    """
    variants = [

word,
# original

word.capitalize(),
# Password

word.upper(),
# PASSWORD

word.lower(),
# password

```

```

word[::-1],
# drowssap

word.translate(_LEET),
# p@$w0rd
    word +
"1", #
password1
    word +
"123", #
password123
    word +
"! ", #
password!
    word +
"2024", #
password2024
    word +
"2025", #
password2025
    "1" +
word, #
1password
    (word[0].upper() + word[1:] + "1") if word
else word, # Password1
    word *
2, #
passwordpassword
]
seen: Set[str] = set()
result: List[str] = []
for v in variants:
    if v not in seen:
        seen.add(v)
        result.append(v)
return result

#
=====
=====
# 3. MARKOV CHAIN GENERATOR

```

```

# Character-level n-gram model trained on common
password patterns.
# Generates statistically plausible candidates –
far smarter than
# alphabetical brute force when the target is a
human-chosen password.
#
=====

_DEFAULT_CORPUS: List[str] = [
    "password", "qwerty", "letmein", "dragon",
    "master", "monkey",
    "sunshine", "princess", "football", "baseball",
    "welcome", "shadow",
    "superman", "batman", "trustno1", "abc123",
    "pass123", "admin",
    "login", "hello", "iloveyou", "mustang", "access",
    "hunter",
    "michael", "jessica", "charlie", "thomas",
    "pepper", "andrew",
    "tiger", "cheese", "buster", "liverpool",
    "thunder", "ranger",
    "silver", "cookie", "matrix", "secret", "dragon",
    "ninja",
]

class MarkovChain:
    """
    Order-n character Markov chain for password-
    pattern generation.

    Training builds transition tables from real-world
    password patterns.
    Generation walks those tables stochastically,
    producing strings that
    follow learned character sequences – much closer
    to human choices than
    exhaustive combinatorics.

    Example (order=2):

```

```

train(["password"]) → chain: "^^"→p, "^p"→a,
"pa"→s, "as"→s ...
generate() → plausible variants
using those transitions
"""

def __init__(self, order: int = 2):
    self.order = order
    self.chain: Dict[str, List[str]] =
defaultdict(list)
    self.starts: List[str] = []

def train(self, corpus: List[str]) ->
"MarkovChain":
    PAD = "^" * self.order
    for word in corpus:
        s = PAD + word + "$"
        # First `order+1` chars = start state
        (e.g. "^^p" for "password")
        self.starts.append(s[: self.order + 1])
        # Build transition table from every n-gram
        → next character
        for i in range(len(s) - self.order):
            self.chain[s[i : i +
self.order]].append(s[i + self.order])
    return self

def train_default(self) -> "MarkovChain":
    """Use the built-in common-password corpus."""
    return self.train(_DEFAULT_CORPUS)

def train_from_file(self, path: str, limit: int =
100_000) -> "MarkovChain":
    """Train on an external wordlist (e.g.
rockyou.txt) for richer patterns."""
    words: List[str] = []
    with open(path, errors="ignore") as f:
        for i, line in enumerate(f):
            if i >= limit:
                break
            w = line.strip()
            if 3 <= len(w) <= 16:

```

```

        words.append(w)
    return self.train(words)

    def _gen_one(self, min_len: int, max_len: int) ->
Optional[str]:
        """Walk the chain once to produce a single
candidate."""
        start = random.choice(self.starts)
        word = start[self.order :]          # strip
padding, keep first char
        state = start[1:]                  # sliding
window for next lookup
        for _ in range(max_len):
            choices = self.chain.get(state)
            if not choices:
                break
            ch = random.choice(choices)
            if ch == "$":
                break
            word += ch
            state = state[1:] + ch
        return word if min_len <= len(word) <= max_len
    else None

    def generate(
        self, n: int = 10_000, min_len: int = 4,
max_len: int = 10
    ) -> List[str]:
        """Generate `n` unique candidates within
[min_len, max_len]."""
        seen: Set[str] = set()
        results: List[str] = []
        attempts = 0
        while len(results) < n and attempts < n * 20:
            attempts += 1
            w = self._gen_one(min_len, max_len)
            if w and w not in seen:
                seen.add(w)
                results.append(w)
        return results

```

```

#
=====

# 4. STATS & LIVE DISPLAY
#
=====

@dataclass
class Stats:
    n_targets: int = 0
    tested: int = 0
    cracked: int = 0
    t0: float = field(default_factory=time.time)

    @property
    def elapsed(self) -> float:
        return time.time() - self.t0

    @property
    def rate(self) -> float:
        return self.tested / max(self.elapsed, 1e-9)

    def render(self, width: int = 30) -> str:
        """Rolling progress bar + live stats for
terminal output."""
        filled = (self.tested % 10_000) * width //
10_000
        bar = "█" * filled + "░" * (width - filled)
        return (
            f"\r [{bar}] {self.rate:>11,.0f} H/s "
            f"tested {self.tested:>11,} "
            f"cracked {self.cracked}/{self.n_targets}"
            "
            f"{self.elapsed:>6.1f}s "
        )

#
=====

# 5. ENGINE

```

```

#
=====

class Engine:
    """
    Multi-modal brute-force engine.

    Architecture
    -----
    Main thread      → pumps candidate batches into a
work queue
    Worker threads  → drain the queue, hash each
candidate, test against targets
    Found lock      → thread-safe result recording;
stops all workers on full crack

    Hybrid pipeline order (most efficient → least
efficient)

    -----

    Phase 1  Raw wordlist          (direct dictionary
hits)
    Phase 2  Wordlist + mutations  (leet, caps,
suffixes, ...)
    Phase 3  Markov + mutations    (statistically
generated candidates)
    Phase 4  Pure brute force      (exhaustive
fallback)
    """

    def __init__(
        self,
        targets: List[str],
        algo: str = "",
        mode: str = "hybrid",
        charset: str = "",
        min_len: int = 1,
        max_len: int = 6,
        threads: int = 0,
        wordlist: str = "",

```

```

        use_rules: bool = True,
        use_markov: bool = True,
        checkpoint: str = "checkpoint.json",
        batch_size: int = 512,
    ):
        self._targets_orig = set(t.lower().strip() for
t in targets)
        self.remaining: Set[str] =
set(self._targets_orig)

        self.algo = algo.lower() if algo else
(auto_detect(targets[0]) if targets else "md5")
        self._hash = get_hash_fn(self.algo)

        self.mode = mode
        self.charset = charset or
(string.ascii_lowercase + string.digits)
        self.min_len = min_len
        self.max_len = max_len
        self.threads = threads or (os.cpu_count() or
4)

        self.wordlist = wordlist
        self.use_rules = use_rules
        self.use_markov = use_markov
        self.checkpoint_path = checkpoint
        self.batch_size = batch_size

        self.found: Dict[str, str] = {} # hash →
plaintext
        self.stats = Stats(n_targets=len(targets))
        self._lock = threading.Lock()
        self._stop = threading.Event()

        self._load_checkpoint()

# — hash testing


---



def _test(self, candidate: str) -> bool:
    """Hash `candidate` and check against
remaining targets."""

```

```

        h = self._hash(candidate)
        if h in self.remaining:
            with self._lock:
                if h in self.remaining:           #
re-check under lock
                    self.remaining.discard(h)
                    self.found[h] = candidate
                    self.stats.cracked += 1
                    print(f"\n ✓ {h[:48]} →
'{candidate}''")
                if not self.remaining:
                    self._stop.set()             #
all targets cracked
                    return True
                return False

# — candidate generators


---



    def _gen_brute(self) -> Generator[str, None,
None]:
        """Pure combinatorial iteration over the
charset."""
        for length in range(self.min_len, self.max_len
+ 1):
            for combo in
itertools.product(self.charset, repeat=length):
                if self._stop.is_set():
                    return
                yield "".join(combo)

    def _gen_wordlist(self) -> Generator[str, None,
None]:
        """Stream candidates from a wordlist file line
by line."""
        p = Path(self.wordlist)
        if not p.exists():
            print(f" [!] Wordlist not found:
{self.wordlist}")
            return
        with p.open(errors="ignore") as f:

```

```

        for line in f:
            if self._stop.is_set():
                return
            yield line.rstrip("\n")

    def _gen_markov(self) -> Generator[str, None,
None]:
        """Markov-generated candidates, optionally
        expanded with mutations."""
        mc = MarkovChain().train_default()
        # Optionally boost quality by training on the
        provided wordlist
        if self.wordlist and
        Path(self.wordlist).exists():
            mc.train_from_file(self.wordlist)
            for w in mc.generate(20_000, self.min_len,
            self.max_len):
                if self._stop.is_set():
                    return
                yield w
            if self.use_rules:
                yield from mutate(w)

    def _gen_hybrid(self) -> Generator[str, None,
None]:
        """
        4-phase pipeline: wordlist → mutations →
        Markov → brute.
        De-duplicates across all phases so no
        candidate is tested twice.
        """
        seen: Set[str] = set()

    def unique(gen: Generator) -> Generator[str,
None, None]:
        for item in gen:
            if self._stop.is_set():
                return
            if item not in seen:
                seen.add(item)
            yield item

```

```

# Phase 1 – raw wordlist
if self.wordlist:
    yield from unique(self._gen_wordlist())

# Phase 2 – wordlist + mutations
if self.use_rules and self.wordlist and
Path(self.wordlist).exists():
    def _wl_mutated() -> Generator[str, None,
None]:
        with
Path(self.wordlist).open(errors="ignore") as f:
            for line in f:
                yield from
mutate(line.rstrip("\n"))
            yield from unique(_wl_mutated())

# Phase 3 – Markov chain + mutations
if self.use_markov:
    mc = MarkovChain().train_default()
    if self.wordlist and
Path(self.wordlist).exists():
        mc.train_from_file(self.wordlist)

    def _markov_with_rules() -> Generator[str,
None, None]:
        for w in mc.generate(20_000,
self.min_len, self.max_len):
            yield w
            if self.use_rules:
                yield from mutate(w)

        yield from unique(_markov_with_rules())

# Phase 4 – brute force fallback
yield from unique(self._gen_brute())

def _pipeline(self) -> Generator[str, None, None]:
    dispatch = {
        "brute": self._gen_brute,
        "wordlist": self._gen_wordlist,
        "markov": self._gen_markov,
        "hybrid": self._gen_hybrid,

```

```

    }
    yield from dispatch.get(self.mode,
self._gen_hybrid)()

# — worker threads


---




---



def _worker(self, q: queue.Queue) -> None:
    while not self._stop.is_set():
        try:
            batch = q.get(timeout=0.3)
        except queue.Empty:
            continue
        if batch is None:          # poison pill –
shut down
            q.task_done()
            break
        for c in batch:
            self._test(c)
            self.stats.tested += 1
        q.task_done()

# — run


---




---



def run(self) -> None:
    self._header()
    wq: queue.Queue =
queue.Queue(maxsize=self.threads * 8)
    pool = [
        threading.Thread(target=self._worker,
args=(wq,), daemon=True)
        for _ in range(self.threads)
    ]
    for t in pool:
        t.start()

    batch: List[str] = []
    try:
        for candidate in self._pipeline():

```

```

        if self._stop.is_set():
            break
        batch.append(candidate)
        if len(batch) >= self.batch_size:
            wq.put(batch)
            batch = []

sys.stdout.write(self.stats.render())
sys.stdout.flush()

    if batch:
        wq.put(batch)

    # Send one poison pill per worker
    for _ in pool:
        wq.put(None)
    wq.join()

except KeyboardInterrupt:
    self._stop.set()
    print("\n\n [!] Interrupted – saving
checkpoint...")
    self._save_checkpoint()

self._summary()

# — benchmark

```

```

def benchmark(self, secs: int = 5) -> None:
    """Show raw hashing throughput for every
    supported algorithm."""
    print(f"\n  Benchmarking all algorithms
    ({secs}s each)...\n")
    print(f"  {'Algorithm':<10}  {'H/sec':>15}
    {'Operations':>14}")
    print(f"  {'-'*10}  {'-'*15}  {'-'*14}")
    sample = "BenchmarkString42!"
    for name, fn in _ALGORITHMS.items():
        n = 0
        deadline = time.time() + secs

```

```

        while time.time() < deadline:
            fn(sample)
            n += 1
            print(f" {name:<10} {n / secs:>15,.0f}
{n:>14,}")
            print()

# — checkpoint


---



def _save_checkpoint(self) -> None:
    data = {
        "tested":    self.stats.tested,
        "found":     self.found,
        "remaining": list(self.remaining),
    }
    with open(self.checkpoint_path, "w") as f:
        json.dump(data, f, indent=2)
    print(f" Checkpoint saved →
{self.checkpoint_path}")

def _load_checkpoint(self) -> None:
    p = Path(self.checkpoint_path)
    if not p.exists():
        return
    try:
        data = json.loads(p.read_text())
        self.found = data.get("found", {})
        self.remaining = set(data.get("remaining",
self._targets_orig))
        self.stats.tested = data.get("tested", 0)
        if self.stats.tested:
            print(f" Resumed –
{self.stats.tested:,} candidates already tested\n")
    except Exception:
        pass

# — display


---



```

```

def _header(self) -> None:
    print(f"""

=====
||  BruteForce Engine  ·  God Tier Edition  ||
=====

Algorithm : {self.algo.upper()}
Mode      : {self.mode}
Threads   : {self.threads}
Targets   : {self.stats.n_targets}
Charset   : {self.charset[:24]}{"..." if
len(self.charset) > 24 else ""}
Range     : {self.min_len}-{self.max_len} chars
""")

    def _summary(self) -> None:
        print(f"\n\n {'=' * 54}")
        print(f"  Targets cracked :
{self.stats.cracked} / {self.stats.n_targets}")
        print(f"  Candidates tried :
{self.stats.tested:,}")
        print(f"  Average rate      :
{self.stats.rate:,.0f} H/s")
        print(f"  Elapsed           :
{self.stats.elapsed:.2f}s")
        if self.found:
            print(f"\n  — Cracked
—————")
            for h, p in self.found.items():
                print(f"    {h[:48]} → '{p}'")
            elif self.stats.cracked == 0:
                print("\n  No targets cracked in this
run.")
            print(f"  {'=' * 54}\n")

#
=====
=====

```

```

# 6. CLI
#
=====

_BANNER = """

=====
||   BruteForce Engine   ·   God Tier Edition   ||
=====
||   brute | wordlist | markov | hybrid         ||
=====
||
=====
"""

def main() -> None:
    parser = argparse.ArgumentParser(
        prog="engine.py",

    formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
examples:
    Crack a single MD5 (auto-detected):
        python engine.py --hash
        5f4dcc3b5aa765d61d8327deb882cf99

    Hybrid attack with rockyou:
        python engine.py --hash <hash> --mode hybrid --
        wordlist rockyou.txt

    Use combinations from generator.py as wordlist:
        python engine.py --hash <hash> --mode wordlist --
        wordlist combinations.txt

    Crack multiple hashes from file:
        python engine.py --hashfile targets.txt --algo
        sha256 --mode hybrid

```

```

Custom charset, longer range:
python engine.py --hash <hash> --mode brute --
charset abcdef0123 --min 4 --max 8

Run hash algorithm benchmark:
python engine.py --benchmark
"""
)

tg = parser.add_argument_group("target")
tg.add_argument("--hash", metavar="H",
help="Single target hash")
tg.add_argument("--hashfile", metavar="FILE",
help="File of hashes, one per line")
tg.add_argument("--algo", metavar="ALGO",
default="",
help="md5 | sha1 | sha256 | sha512
| ntlm (auto-detected if omitted)")

ag = parser.add_argument_group("attack")
ag.add_argument("--mode", default="hybrid",
choices=["brute", "wordlist",
"markov", "hybrid"],
help="Attack mode (default:
hybrid)")
ag.add_argument("--wordlist", metavar="FILE",
default="",
help="Path to wordlist / use
combinations.txt from generator.py")
ag.add_argument("--charset", metavar="STR",
default="",
help="Override charset for brute
mode (default: a-z + 0-9)")
ag.add_argument("--min", type=int, default=1,
metavar="N", help="Min candidate length")
ag.add_argument("--max", type=int, default=6,
metavar="N", help="Max candidate length")
ag.add_argument("--no-rules", dest="rules",
action="store_false",
help="Disable mutation rule
engine")

```

```

        ag.add_argument("--no-markov", dest="markov",
            action="store_false",
            help="Disable Markov chain generator")

        pg = parser.add_argument_group("performance")
        pg.add_argument("--threads", type=int, default=0,
            metavar="N",
            help="Worker threads (default: CPU core count)")
        pg.add_argument("--batch", type=int,
            default=512, metavar="N",
            help="Candidates per work unit (default: 512)")

        mg = parser.add_argument_group("misc")
        mg.add_argument("--checkpoint",
            default="checkpoint.json",
            help="Checkpoint file for pause/resume (default: checkpoint.json)")
        mg.add_argument("--benchmark",
            action="store_true",
            help="Benchmark all hash algorithms and exit")

        args = parser.parse_args()
        print(_BANNER)

        if args.benchmark:
            Engine([], "md5").benchmark()
            return

        targets: List[str] = []
        if args.hash:
            targets.append(args.hash.strip())
        if args.hashfile:
            fp = Path(args.hashfile)
            if not fp.exists():
                sys.exit(f" [!] Hash file not found: {args.hashfile}")
            targets += [ln.strip() for ln in fp.read_text().splitlines() if ln.strip()]

```

```
if not targets:
    parser.error("Provide --hash or --hashfile
(or --benchmark to test speed)")

Engine(
    targets    = targets,
    algo       = args.algo,
    mode       = args.mode,
    charset    = args.charset,
    min_len    = args.min,
    max_len    = args.max,
    threads    = args.threads,
    wordlist    = args.wordlist,
    use_rules   = args.rules,
    use_markov  = args.markov,
    checkpoint  = args.checkpoint,
    batch_size  = args.batch,
).run()

if __name__ == "__main__":
    t0 = time.time()
    main()
    print(f" Wall time: {time.time() - t0:.2f}s\n")
```